

ComEng - Ergänzung

FS 2026 – Prof. Lars Kamm

Autoren:
Mirko Ratti

Version:
1.0.20260605

<https://github.com/mirkoratti/FS2026>



Inhaltsverzeichnis

1 Hex - Dez	2	7 Optimierung und Datenstrukturen	2
2 Instruction Set Architecture (ISA)	2	8 Vergleich ARM Cortex Architekturen	3
3 Logik der ALU und Program Status Register	2	8.1 Evolution Cortex-M0 vs M0+	3
3.1 Flag-Mechanismus (APSR)	2	9 Adressierungsarten ARM (Cortex-M0)	3
4 Systemkomponenten (SCS)	2	9.1 Register Direkt, Immediate	3
4.1 Fault- und Ausnahmemechanismen	2	9.2 Register Indirekt (ptr)	3
4.2 System Tick Timer (SysTick)	2	9.3 PC-Relative & Literal Pool	3
5 Stack und Speicherverwaltung	2	9.4 Registerlisten & Stack (SP-Relative)	3
6 Interrupt-Verwaltung (NVIC)	2	9.5 Stack Framing	3
6.1 NVIC Special Features	2	10 Der Kompilierungsprozess	3
6.2 Maskierungsregister	2	10.1 Compiler-Analyse	3
6.3 Latenz und Stacking	2	11 I/O-Schnittstellen	3
6.4 Preemption vs Subpriority	2	12 Beispiele	3
6.5 Alternative Lösungen & IRQ-Signale	2		

- **Iterationen (Loop):** Es wird ein **Sprung nach hinten** am Ende der Schleife verwendet, wobei die **Assembly-Bedingung wie in C** beibehalten wird (Sprung zum Anfang, wenn die Bedingung noch wahr ist).

7.0.3 Effizienz der Datenstrukturen

- **Lokale Arrays:** Sehr **ineffizient**. hohe Anzahl von Zyklen nur für die Initialisierung im Stack-Frame
- **Strukturen:** Es ist empfehlenswert, die **Strukturen global zu definieren**. Die lokale Initialisierung auf dem Stack führt zu einem hohen Overhead an Taktzyklen.
- **Bitfields:** Extrem **ineffizient**. Obwohl sie Speicher sparen, erfordern sie komplexe Operationen von *Read-Modify-Write* (LDR, Shift, Mask, OR, STR)

8 Vergleich ARM Cortex Architekturen

Cortex-A (Application): Generische Berechnung mit hoher Leistung (Linux/Android).
Cortex-R (Real-Time): Geringe Latenz und hohe Zuverlässigkeit (Automotive/Medizin).
Cortex-M (Microcontroller): Reduzierter Energieverbrauch, deterministische Interrupts.

Core	Thumb	Thumb-2	HW Mult.	HW Div.	Math Sat.	DSP	FPU	Arch
Cortex-M0	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M0+	Mostly	Subset	1/32 cyc	No	No	No	No	ARMv6-M
Cortex-M1	Mostly	Subset	3/33 cyc	No	No	No	No	ARMv6-M
Cortex-M3	Fully	Fully	1 cyc	Yes	Yes	No	No	ARMv7-M
Cortex-M4	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M
Cortex-M7	Fully	Fully	1 cyc	Yes	Yes	Yes	Optional	ARMv7E-M

8.1 Evolution Cortex-M0 vs M0+

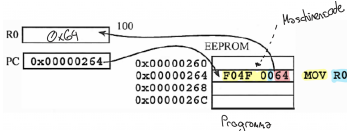
- **M0+ Pipeline:** Auf 2 Stufen reduziert für höhere Effizienz.
 - **I/O-Zugriff:** Unterstützung für Single-Cycle GPIO.
 - **Debugging:** Optionaler Micro Trace Buffer (MTB) zum Nachverfolgen von Sprüngen.
- Pipeline**
+ Es wird in jedem Cycle ein Befehl ausgeführt
- Bei Sprüngen muss die Pipeline 'geleert' werden
- MPU**

9 Adressierungsarten ARM (Cortex-M0)

- Register direkt
- Register Indirekt
- Register Indirekt mit offset [R?, #imm]
- Register Indirekt mit index [R?, R?]

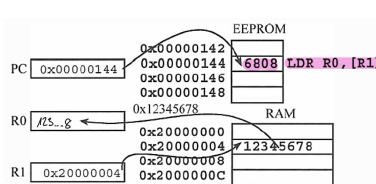
9.1 Register Direkt, Immediate

- **Immediate:** MOV R0, #100 → Der Wert wird direkt geladen.
- **Register:** MOVS R2, R1 → Kopiert den Wert zwischen Registern.
- **Einschränkungen:** Die Immediate-Werte werden in der Instruktion kodiert (z.B. 8-bit für MOVS).



9.2 Register Indirekt (ptr)

- **Einfach:** LDR R0, [R1] → R0 = Wert an der Adresse R1.
- **Mit Offset:** LDR R0, [R1, #4] → R0 = Wert an der Adresse R1 + 4 (nützlich für Arrays/Structs).
- **Mit Index:** LDR R0, [R1, R2] → R0 = Wert an der Adresse R1 + R2.



9.3 PC-Relative & Literal Pool

- **Problem:** 32-bit-Instruktionen können keine 32-bit-Konstanten enthalten.
- **Lösung:** Der Assembler speichert die Konstante im **Literal Pool** (Speicherbereich in der Nähe).
- **Zugriff:** LDR R0, =0x12345678 wird übersetzt in LDR R0, [PC, #offset].
- Es sind immer **zwei Speicherzugriffe** erforderlich

9.4 Registerlisten & Stack (SP-Relative)

- **Multiple:** LDM R7!, {R0-R3, R5} → Lädt mehrere Register beginnend bei der Adresse in R7.
- **Stack:** PUSH {R0-R2}, POP {R0-R2}.
- **Goldene Regel:** Das Register mit der niedrigsten Nummer (R0) geht immer in die niedrigste Speicheradresse. Die Reihenfolge in den geschweiften Klammern ist für die Hardware nicht relevant.

9.5 Stack Framing

Der Stack Frame ist der in RAM reservierte Bereich für lokale Variablen einer Funktion.

- **Allokation:** SUB SP, SP, #X reserviert X Byte (der Stack wächst zu niedrigeren Adressen).
- **Frame Pointer (R7):** Man verwendet MOV R7, SP, um einen festen Bezug zum Anfang des Frames zu haben.
- **Zugriff:** Die Variablen werden unter Verwendung von Offsets relativ zu R7 geschrieben/gelesen, z.B.: STR R0, [R7, #4].
- **Deallokation:** Vor dem Rücksprung (BX LR) muss der Stack mit ADD SP, SP, #X bereinigt werden.

C-Code	Assembly
<pre>void main(void) { uint32_t a = 100; int8_t b = -5; // user code }</pre>	<pre>main: 1 SUB SP, SP, #8 2 MOV R7, SP 3 MOVS R0, #100 4 MVNS R1, #4 5 STR R0, [R7, #0] 6 STRB R1, [R7, #4] 7 ; user code 8 ADD SP, SP, #8 9 BX LR</pre>

10 Der Kompilierungsprozess

Der Quellcode wird durch verschiedene Phasen in Binärdaten umgewandelt:

1. **Präprozessor:** Verarbeitet Einbindungen und Makros. (#define, #if, ...)
2. **Compiler:** Übersetzt die Sprache auf hoher Ebene in Assembly.
3. **Assembler:** Konvertiert Assembly in relocierbaren Maschinencode.
4. **Linker/Binder:** Verbindet Objektdateien und Bibliotheken.
5. **Loader:** Konvertiert relocierbare Adressen in absolute und lädt sie in den Speicher.

10.1 Compiler-Analyse

- **Lexikalische Analyse:** Entfernen von Leerzeichen und Gruppierung von Tokens.
- **Syntaktische/Semantische Analyse:** Prüfung der Struktur und der Datentypen.
- **Optimierung:** Verbesserung des Codes hinsichtlich Geschwindigkeit oder Speicherplatz.
- **Generierung:** Erzeugung des Zielcodes.

11 I/O-Schnittstellen

Die Peripheriegeräte werden über spezifische Register in den Speicher abgebildet (*Memory Mapped I/O*):

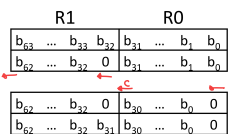
- **Datenregister:** Enthält die zu verarbeitenden Daten.
- **Steuerregister:** Konfiguration des Peripheriegeräts.
- **Statusregister:** Zeigt den aktuellen Zustand des I/O (z. B. Busy, Ready) an.

12 Beispiele

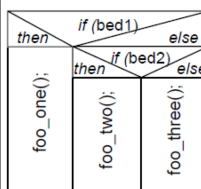
```
1 ;Example: 64Bit Addition
2 LDR r4, =addrOp ;load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ;load op1[r1,r0] and op2[r2,r3]
4 ADDS r0, r0, r1 ;add lower words and set carry
5 ADCS r1, r2, r3 ;add higher words with carry in
6 ADDS r4, #8 ;set r4 to address of res
7 STM r4, {r0-r1} ;store 64bit result from r0, r1 to res
```

```
1 ; Example: 64Bit Subtraction
2 LDR r4, =addrOp ; load address of int64 array [op1,op2,res]
3 LDM r4, {r0-r3} ; load op1[r1,r0] and op2[r3,r2]
4 SUBS r0, r0, r2 ; sub lower words and set borrow
5 SBCS r1, r1, r3 ; sub higher words with borrow in
6 ADDS r4, #16 ; set r4 to address of res
7 STM r4!, {r0-r1} ; store 64bit result from r0, r1 to res
```

```
1 ;Example for 64bit logical shift left
2 LDR r2, =data64 ;load address of data64
3 LDM r2, {r0-r1} ;load data64 to r0, r1
4 LSLS r1, r1, #1 ;logical shift left r1 by 1
5 MOVS r3, #0 ;r3 = 0
6 LSLS r0, r0, #1 ;lsl r0 by 1 (carry = b31)
7 ADCS r1, r3 ;r1 = r1 + 0 + carry
8 STM r2!, {r0-r1} ;store r0,r1 in data64
```



Äquivalent zu: (·2)



Thumb-2 Assembly	C Source Code
<pre>LDR R0, =G2 LDR R1, =G1 LDRH R0, [R0] ;R0 <- G2 SKTH R0, R0 LDR R1, [R1] ;R1 <- G1 CMP R0, R1 ;G2-G1 BGE else_if then: BL foo_one B endif else_if: BL foo_two B endif else: BL foo_three endif:</pre>	<pre>uint8_t G1; // signed 32bit uint8_t G2; // signed 16bit ... if(G1 > G2) // G2-G1 < 0? { foo_one(); } else if(G1 < G2) // G2-G1 > 0? { foo_two(); } else // G2 == G1 { foo_three(); }</pre>

Thumb-2 Assembly	C Source Code
<pre>0x080000F6 LDR R1, =State 0x080000F8 LDRB R0, [R1] 0x080000FA CMP R0, #3 0x080000FC BHI default 0x08000100 LSLS R0, R0, #2 0x08000102 ADR R1, lookup 0x08000104 LDR R0, [R1, R0] 0x08000106 MOV PC, R0 ;lookup 0x08000108 .word 0x08000130 ;addr of case 0 0x0800010C .word 0x08000136 ;addr of case 1 0x08000110 .word 0x0800013C ;addr of case 2 0x08000114 .word 0x08000142 ;addr of case 3 ... case_0: 0x08000130 BL foo_one 0x08000134 B end_switch case_1: 0x08000136 BL foo_two 0x0800013A B end_switch case_2: 0x0800013C BL foo_three 0x08000140 B end_switch case_3: 0x08000142 BL foo_four 0x08000146 B end_switch default: 0x08000148 MOVS R0, #0 0x0800014A STRB R0, [R1] end_switch: 0x0800014C -</pre>	<pre>uint8_t State; // unsigned 8bit ... switch(State) { case 0: foo_one(); break; case 1: foo_two(); break; case 2: foo_three(); break; case 3: foo_four(); break; default: State = 0; break; } -</pre>

1. **Kontrolle der Grenzen (Bound Check):** Anfangsprüfung des Werts von State, um sofort zum default zu springen, falls der Index außerhalb des zulässigen Bereichs liegt.
2. **Berechnung des Offsets:** Multiplikation des Werts mit 4 (State << 2) mittels logischem Shift, da jeder Zeiger in der Look-up-Tabelle 32 Bit (4 Byte) belegt.
3. **Abruf des Zeigers:** Lesen der exakten Zieladresse, indem der berechnete Offset zur Basisadresse der Tabelle addiert wird.
4. **Sprung (Branch):** Laden der abgerufenen Adresse in den Program Counter (PC), um die Ausführung des spezifischen Falls zu starten.